

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 3_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Point;

int orientation(Point a, Point b, Point c) {
    int val = (b.y - a.y) * (c.x - b.x) -
              (b.x - a.x) * (c.y - b.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

int compare(const void *p1, const void *p2) {
    Point *a = (Point *)p1;
    Point *b = (Point *)p2;
    if (a->x == b->x) return a->y - b->y;
    return a->x - b->x;
}

int onSegment(Point a, Point b, Point c) {
    return (c.x <= (a.x > b.x ? a.x : b.x) && c.x >= (a.x < b.x ? a.x : b.x) &&
           c.y <= (a.y > b.y ? a.y : b.y) && c.y >= (a.y < b.y ? a.y : b.y));
}

int pointInConvexHull(Point hull[], int m, Point p) {
    for (int i = 0; i < m; i++) {
        Point a = hull[i];
        Point b = hull[(i + 1) % m];
        int orient = orientation(a, b, p);
        if (orient == 1) return 0;
    }
    return 1;
}

int main() {
    int N;
    scanf("%d", &N);
```

```

Point points[N];
for (int i = 0; i < N; i++) {
    scanf("%d %d", &points[i].x, &points[i].y);
}
Point query;
scanf("%d %d", &query.x, &query.y);

qsort(points, N, sizeof(Point), compare);

Point hull[2*N];
int m = 0;

for (int i = 0; i < N; i++) {
    while (m >= 2 && orientation(hull[m-2], hull[m-1], points[i]) != 2) m--;
    hull[m++] = points[i];
}
for (int i = N-2, t = m+1; i >= 0; i--) {
    while (m >= t && orientation(hull[m-2], hull[m-1], points[i]) != 2) m--;
    hull[m++] = points[i];
}
m--;

if (pointInConvexHull(hull, m, query))
    printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
else
    printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted

centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each. $1e-9$ is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

Output Format

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 0

2 0

2 2

0 2

1 1

Output: Weighted Centroid: 1.00 1.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point pivot;
```

```
double distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
int orientation(Point p, Point q, Point r) {  
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);  
    if (fabs(val) < 1e-9) return 0;  
    return (val > 0) ? 1 : 2;  
}
```

```

int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point *pts = (Point *)malloc(n * sizeof(Point));
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
    }

    if (n < 3) return 0;

    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }

    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;

    pivot = pts[0];
    qsort(&pts[1], n - 1, sizeof(Point), compare);

    int m = 1;
    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
        pts[m++] = pts[i];
    }

    if (m < 3) return 0;

    Point *hull = (Point *)malloc(n * sizeof(Point));

```

```

int hullSize = 0;
hull[hullSize++] = pts[0];
hull[hullSize++] = pts[1];
hull[hullSize++] = pts[2];

for (int i = 3; i < m; i++) {
    while (hullSize >= 2 && orientation(hull[hullSize - 2], hull[hullSize - 1], pts[i]) !=
2)
        hullSize--;
    hull[hullSize++] = pts[i];
}

double sumX = 0, sumY = 0;
for (int i = 0; i < hullSize; i++) {
    sumX += hull[i].x;
    sumY += hull[i].y;
}

printf("Weighted Centroid: %.2f %.2f\n", sumX / hullSize, sumY / hullSize);

free(pts);
free(hull);
return 0;
}

```

Status : Partially correct

Marks : 6/10

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input

points.hull[100000]: An array that can store up to 100,000 points on the convex hull.
1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0). Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull. Convex Hull Points: (0, 0), (3, 0), (3, 2), (0, 2). Perimeter Calculation:

Distances between consecutive hull points: (0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00. Total Perimeter: 10.00.

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

Output Format

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output: Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#define MAXN 100000
```

```
#define EPS 1e-9
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point points[MAXN], hull[MAXN];  
int N, hullSize;  
Point pivot;
```

```
double cross(Point p1, Point p2, Point p3) {  
    return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);  
}
```

```
double dist(Point a, Point b) {  
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));  
}
```

```
int cmp(const void *a, const void *b) {  
    Point *p1 = (Point *)a;  
    Point *p2 = (Point *)b;  
    double c = cross(pivot, *p1, *p2);  
    if (fabs(c) < EPS) {  
        double d1 = (p1->x - pivot.x) * (p1->x - pivot.x) + (p1->y - pivot.y) * (p1->y - pivot.y);  
        double d2 = (p2->x - pivot.x) * (p2->x - pivot.x) + (p2->y - pivot.y) * (p2->y - pivot.y);  
        return d1 < d2 ? -1 : 1;  
    }  
    return c > 0 ? -1 : 1;  
}
```

```
void convexHull() {  
    pivot = points[0];  
    int pivotIndex = 0;  
    for (int i = 1; i < N; i++) {  
        if (points[i].y < pivot.y || (fabs(points[i].y - pivot.y) < EPS && points[i].x < pivot.x)) {  
            pivot = points[i];  
            pivotIndex = i;  
        }  
    }  
    Point temp = points[0];  
    points[0] = points[pivotIndex];
```

```

points[pivotIndex] = temp;
qsort(points + 1, N - 1, sizeof(Point), cmp);
hull[0] = points[0];
hull[1] = points[1];
hullSize = 2;
for (int i = 2; i < N; i++) {
    while (hullSize >= 2 && cross(hull[hullSize - 2], hull[hullSize - 1], points[i]) <=
0) {
        hullSize--;
    }
    hull[hullSize++] = points[i];
}

double perimeter() {
    double p = 0;
    for (int i = 0; i < hullSize; i++) {
        p += dist(hull[i], hull[(i + 1) % hullSize]);
    }
    return p;
}

int main() {
    scanf("%d", &N);
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    convexHull();
    printf("Convex Hull:\n");
    for (int i = 0; i < hullSize; i++) {
        printf("%.0f %.0f\n", hull[i].x, hull[i].y);
    }
    double p = perimeter();
    printf("Perimeter: %.2f\n", p);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays. $1e-9$: Threshold for floating-point equality checks to handle precision errors. $1e-12$: Threshold to detect degenerate (zero-length) segments. $1e30$: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
void rotate(int** matrix, int size) {
    for (int i = 0; i < size / 2; i++) {
        for (int j = i; j < size - i - 1; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[size - 1 - j][i];
            matrix[size - 1 - j][i] = matrix[size - 1 - i][size - 1 - j];
            matrix[size - 1 - i][size - 1 - j] = matrix[j][size - 1 - i];
            matrix[j][size - 1 - i] = temp;
        }
    }
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    rotate(matrix, n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    for (int i = 0; i < n; i++) free(matrix[i]);
    free(matrix);

    return 0;
}
```

Status : Wrong

Marks : 0/10

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays. $1e-9$: Threshold for floating-point equality checks to handle precision errors. $1e-12$: Threshold to detect degenerate (zero-length) segments. $1e30$: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2
3 2

Output: 2 2

3 2
1 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
typedef struct {
    double x, y;
} Point;
```

```
typedef struct {
    Point p;
    double dist;
    int id;
} PointDist;
```

```
Point pivot;
```

```
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}
```

```
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
int comparePoints(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
```

```

int compareDist(const void* a, const void* b) {
    PointDist* pd1 = (PointDist*)a;
    PointDist* pd2 = (PointDist*)b;
    if (fabs(pd1->dist - pd2->dist) > 1e-9) {
        return (pd1->dist > pd2->dist) ? -1 : 1;
    }
    return pd1->id - pd2->id;
}

```

```

double distToSegment(Point p, Point a, Point b) {
    double dx = b.x - a.x;
    double dy = b.y - a.y;
    double l2 = dx * dx + dy * dy;
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * dx + (p.y - a.y) * dy) / l2;
    if (t < 0) t = 0;
    else if (t > 1) t = 1;
    Point proj = {a.x + t * dx, a.y + t * dy};
    return sqrt(distSq(p, proj));
}

```

```

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2) return 0;

```

```

    Point pts[n], original[n];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        original[i] = pts[i];
    }

```

```

    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }

```

```

    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;

    pivot = pts[0];

```

```

qsort(&pts[1], n - 1, sizeof(Point), comparePoints);

int m = 1;
for (int i = 1; i < n; i++) {
    while (i < n - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
    pts[m++] = pts[i];
}

Point stack[n];
int top = 0;
if (m >= 1) stack[top++] = pts[0];
if (m >= 2) stack[top++] = pts[1];
if (m >= 3) stack[top++] = pts[2];

for (int i = 3; i < m; i++) {
    while (top > 1 && orientation(stack[top - 2], stack[top - 1], pts[i]) != 2) top--;
    stack[top++] = pts[i];
}

PointDist res[n];
for (int i = 0; i < n; i++) {
    double minDist = 1e30;
    for (int j = 0; j < top; j++) {
        double d = distToSegment(original[i], stack[j], stack[(j + 1) % top]);
        if (d < minDist) minDist = d;
    }
    res[i].p = original[i];
    res[i].dist = minDist;
    res[i].id = i;
}

qsort(res, n, sizeof(PointDist), compareDist);

for (int i = 0; i < k; i++) {
    printf("%d %d\n", (int)res[i].p.x, (int)res[i].p.y);
}

return 0;
}

```

Status : Partially correct

Marks : 4/10